AA 272 Fall 2025. Homework 1

YOUR NAME, YOUR_EMAIL@stanford.edu

Instructors: Prof. Grace Gao, Keidai Iiyama, Guillem Casadesus Vila, and Hannah Nabavi

September 26, 2025

Goal of this problem set: GPS has three segments: Control, Space, and User. In this problem set you will learn a bit about each segment and the background information that you will need for the rest of the term. In the last problem of the problem set, you will think critically about position, navigation, and timing requirements through the lens of a systems engineer scoping out their GPS project.

Instructions & Notes

1. This problem set is **due at 4:00 PM on October 3, 2025 via Gradescope**.
2. This homework is scored out of **50 points total** and includes a corresponding self-care component. The point value for each homework question is indicated next to the question number. Note that each homework will be equally weighted in the calculation of your final grade.
3. Please note that homework is intended to supplement and reinforce your learning. We design them to guide you through the process of going deeper into the concepts we cover in class, and beyond.
4. You are free to discuss class contents and these problems with others. However, your submitted solutions and code must be your own work, written from scratch.
5. **AI tools** (e.g., Chat GPT, Gemini) can be viewed as search engines or even classmates. They must not be used to directly solve any of the problems. For example, it is acceptable to discuss general topics about GNSS, edit a plot, or how so solve a linear system using `numpy`.
6. **Ensure that you typeset your solutions**. We encourage using the provided notebooks or LaTeX to type up your solutions, but you are free to use Microsoft Word or other editors.
7. Ensure that your solution includes the plots required by the question, **in the specified format** outlined in the problem for full credit. **Please submit your code in the programming assignment entry on Gradescope**. You can submit Colab notebooks and other interactive formats as links in a `.txt` format.
8. If you are spending an excessive amount of time on any part of any question, please post on Ed or come to office hours (see Canvas).
9. Please **start early** so you can address any questions you might have, and have fun!

Problem 1: GPS Control Segment [8 points + 1 bonus]

Goal of this problem: Students will learn about the GPS control segment and services that routinely provide information about GPS. As a bonus, students will learn more about the initial design of GPS.

Familiarize yourself with some excellent reference material about GPS before answering the questions below. Particularly useful sites include:

- http://www.navcen.uscg.gov/ - The U.S. Coast Guard Navigation Center (NAVCEN). Good site for technical information on GPS.
- http://www.ngs.noaa.gov/CORS/ - The Continuously Operating Reference Stations (CORS) site maintained by the National Geodetic Survey. Data from several hundred reference GPS stations across the U.S. Also has links detailing several data formats associated with GPS.
- https://www.igs.org/ - The International GNSS Service (IGS). Contains data and products relating to the precise locations of each GPS satellite.

These resources are useful for extracting publicly available information about the system and might be useful during final projects as well.

1.1 [2 points]

Using information from the CORS website, under the NCN Map tab, find the identifier for the closest CORS receiver to Stanford campus. Report its name and **antenna reference point** coordinates (latitude, longitude, height from reference ellipsoid) in the ITRF 2020 reference frame. You can leave the latitude and longitude coordinates in degrees, minutes, and seconds, i.e., the format provided by the website.

ANSWER

- Station Name: SLAC_BARD_CN2002
- Latitude: 37º 24' 59.46807"
- Longitude: -122º 12' 15.36454"
- Ellipsoid: 63.667

1.2 [2 points]

Referring to the CORS website again, find the GPS week and day of the year for September 4, 2025 (5th anniversary of the Lonely Halls Meeting where GPS was initially architected). Use the GPS Calendar tab for this problem.

ANSWER

- GPS Week: 2382
- Day of year: 247

1.3 [4 points]

One important role of the control segment is to monitor anomalies (such as interference from human and natural sources) that affect the accuracy and availability of the navigation signal.

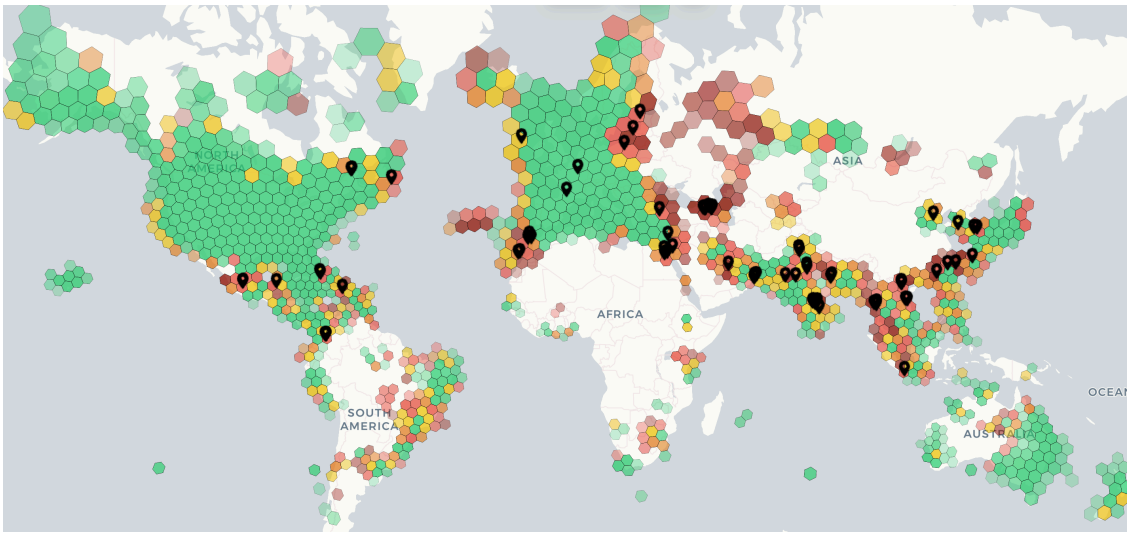
1.3.1 [2 points] Refer to Stanford's GNSS jamming map at https://rfi.stanford.edu/. The website was developed by Ph.D. students in the GPS Lab and shows a global map where areas with high GNSS interference are highlighted in red. Take a screenshot of the map. In what regions do we observe high interference? What are the possible causes of high interference that you observe on the map?

[24]:
```
from google.colab import drive
drive.mount("/content/drive")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
```

[25]:
```
from IPython.display import Image
Image("/content/drive/MyDrive/AA 272 - GPS Class/HW1/figures/jamming_map.png",␣
↪width=800)
```

[25]:



- On the coast of China
- In the Middle East
  - Qatar
  - Palestine/Israel/Jordan/Lebanon
  - Turkey (near Istanbul)
  - Armenia
- Morocco

The jammed regions seem to be in somewhat militarized regions, like near Palestine and Israel, and in the South China Sea (which is highly disputed by China).

1.3.2 [2 points] One cause of high interference is the delay caused by the electron content in the ionosphere. A website from JPL: https://guardian.jpl.nasa.gov/analysis/globalIonoMovie/index.html shows the global map of

total electron content (TEC). Take a screenshot of the map. In what regions do you observe high TEC? How does the TEC change over a day?

ANSWER

[26]:
```
Image("/content/drive/MyDrive/AA 272 - GPS Class/HW1/figures/ion_content_map.
↪png", width=800)
```

```
Output hidden; open in https://colab.research.google.com to view.
```

The highest TEC appears around the equator, during the daytime. Outside this region, and at night, TEC is similar to the rest of the world, which has a low TEC.

1.4 Bonus [1 point]

Watch the following documentary: *The Story of GPS: "The Lonely Halls Meeting,"* which is available on YouTube (free). Then, list at least four challenges that the designers had to overcome and the timestamp of the moment that the challenge was discussed in the video. Note that retired Colonel Gaylord Green (our week 9 guest lecturer) is featured throughout the documentary.

YouTube Link: https://www.youtube.com/watch?v=Z5N4CqJLAhQ

For more on Stanford's history with GPS, please see the following link: https://scpnt.stanford.edu/about-scpnt/scpnt-history

ANSWER

TODO

Challenge 1

- Timestamp: 17:00
- Description: Lot of space junk exists in LEO, so having satellites in MEO and GEO allows them to be in less danger

Challenge 2

- Timestamp: 17:40
- Description: Geometric Dilution of Precision is introduced if satellites are all in a line (e.g. all in GEO), so multiple different orbital planes were used

Challenge 3

- Timestamp: 28:42
- Description: They didn't want to use 24 different channels, so they created Code Division Multiple Access

Challenge 4

- Timestamp: 32:00
- Description: They needed a small atomic clock, when previous ones were the size of big microwaves, so they enlisted the help of Ernst Jechart to make a 4" x 4" x 4" atomic oscillator

Problem 2: GPS Space Segment [14 points]

Goal of this problem: Students will learn about the GPS space segment, GNSS orbits, and the effect on receivers at different latitudes. Students will also get exposure to the open-source GNSS Python Library (`gnss-lib-py`) developed and maintained by the NAV Lab here at Stanford. The GNSS Python Library is registered in the Python package index (https://pypi.org/project/gnss-lib-py/). So, you can install it by simply typing the following in the first cell:

```
%pip install gnss-lib-py
```

If you are *new to Google Colab*, please start this early and ask any questions so that we can resolve any issues before the due date.

Behind the scenes in this problem, we are querying the IGS (see problem 1) precise ephemeris data. This post-processed data (roughly weekly) contains the control segment's "best" estimates of the satellite positions. IGS compiles estimates for multiple navigation satellite constellations. You will notice the following: GPS (US), GLONASS (Russia), Galileo (EU), and Beidou (China). But, there is also one regional navigation satellite system in this problem: QZSS. This is the "Quasi-Zenith Satellite System" from JAXA in Japan. Unfortunately, this problem does not include IRNSS (Indian Regional Navigation Satellite System), also known as NavIC (Navigation with Indian Constellation). See more at https://www.gps.gov/systems/gnss/.

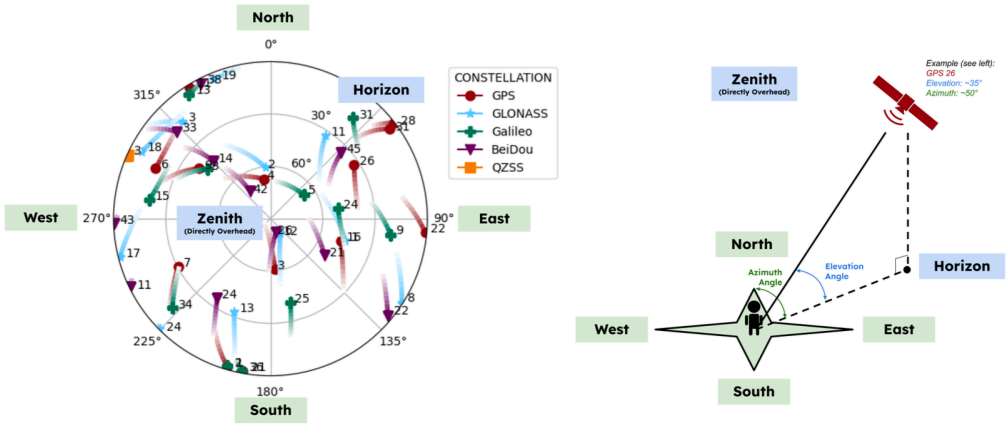
Lastly, in this problem, we are making skyplots. This is a term in the GPS/GNSS community referring to a hemisphere representation of which satellites you would see (i.e., which satellites you will get a signal from and which direction the signal would come from).

Figure: Skyplot (left) and corresponding geometric interpretation (right)

[20]:
```
from google.colab import drive
drive.mount("/content/drive")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
```

[21]:
```
from IPython.display import Image, display

display(Image(filename="/content/drive/MyDrive/AA 272 - GPS Class/HW1/figures/
 ↪SkyplotUnderstanding.png", width=800))
```



2.1 [2 points]

Follow the tutorial at https://gnss-lib-py.readthedocs.io/en/latest/tutorials/utils/tutorials_ephemeris_downloade
Include the skyplot showing the satellite elevation and azimuth over one hour.

Hint: This plot should exactly match the tutorial.

Note: For the rest of this problem, please feel free to use the same code as from the tutorial, but change the input latitude, longitude, altitude, start time, and end time.

ANSWER

[22]:
```
%pip install gnss-lib-py > /dev/null
```

[23]:
```
import numpy as np
import gnss_lib_py as glp
from datetime import datetime, timezone, timedelta
from zoneinfo import ZoneInfo
from pytz import all_timezones
```

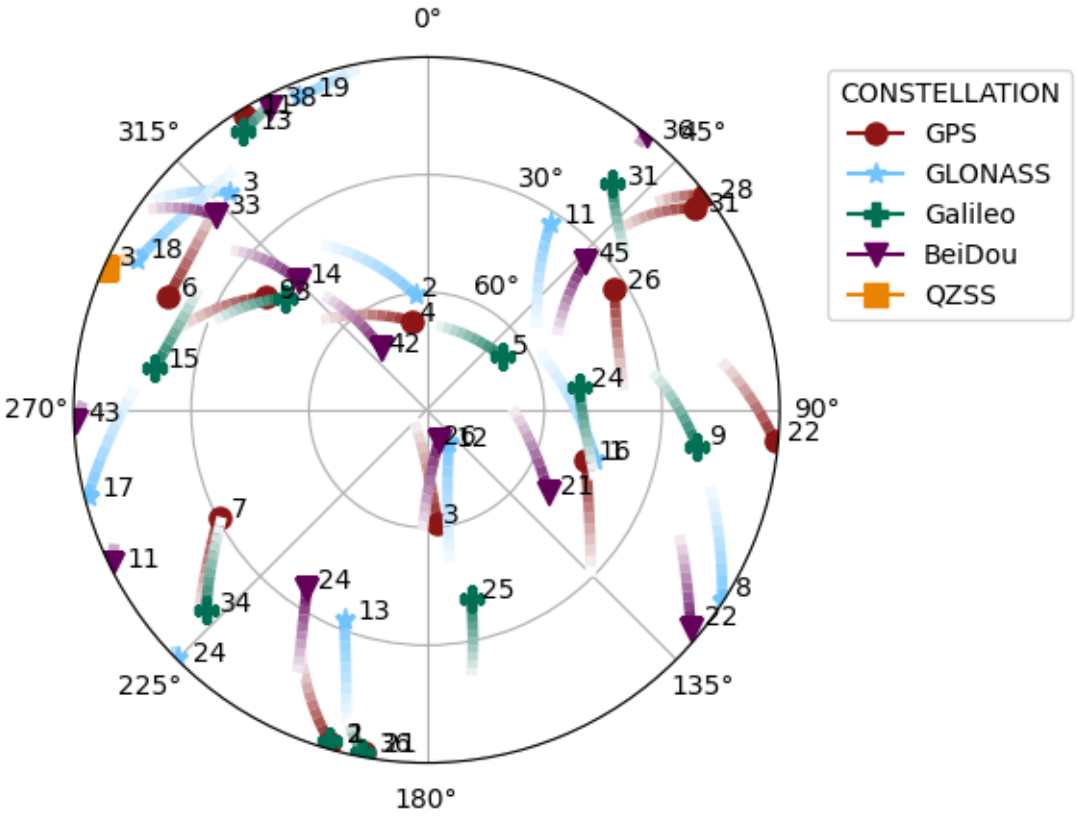
[24]:
```
def plot_gps(lat, lon, alt, timestamp_start, timestamp_end, *, where_key_idx =␣
 ↪None, where_value = None):
  gps_millis = glp.datetime_to_gps_millis(np.
 ↪array([timestamp_start,timestamp_end]))
  sp3_path = glp.load_ephemeris(file_type="sp3",
                                gps_millis=gps_millis,
                                verbose=True)

  sp3 = glp.Sp3(sp3_path)

  # create receiver state NavData instance to pass into skyplot function
  x_rx_m, y_rx_m, z_rx_m = glp.geodetic_to_ecef(np.array([[lat,lon,alt]]))[0]
  receiver_state = glp.NavData()
```

```
  receiver_state["gps_millis"] = glp.datetime_to_gps_millis(timestamp_start)
  receiver_state["x_rx_m"] = x_rx_m
  receiver_state["y_rx_m"] = y_rx_m
  receiver_state["z_rx_m"] = z_rx_m

  cropped_sp3 = sp3.where("gps_millis",gps_millis[0],"geq").
 ↪where("gps_millis",gps_millis[1],"leq")

  if where_key_idx is not None and where_value is not None:
    cropped_sp3 = cropped_sp3.where(where_key_idx, where_value)

  fig = glp.plot_skyplot(cropped_sp3,receiver_state)
```

[25]:
```
lat, lon, alt = 37.42984154652992, -122.16946303566934, 0.
timestamp_start = datetime(year=2023, month=3, day=14, hour=12, tzinfo=timezone.
 ↪utc)
timestamp_end = datetime(year=2023, month=3, day=14, hour=13, tzinfo=timezone.
 ↪utc)
```

[26]:
```
plot_gps(lat, lon, alt, timestamp_start, timestamp_end)
```

```
ephemeris dates needed: [datetime.date(2023, 3, 14)]
using previously downloaded file:
 /content/data/ephemeris/sp3/COD0MGXFIN_20230730000_01D_05M_ORB.SP3
using previously downloaded file:
 /content/data/ephemeris/sp3/COD0MGXFIN_20230730000_01D_05M_ORB.SP3
```



[Replace with your answer]

2.2 [2 Point]

Use the latitude, longitude, and height above the ellipsoid (called altitude in the code) of the SLAC station from problem 1.1. Feel free to use Wolfram Alpha or another online conversion tool to get the coordinates in decimal form. Use 9 am - 10 am Pacific Time on September 22, 2025 as the query time (the first day of this quarter).

Hint: Note that longitude at SLAC is *negative* since we are *West* of the prime meridian.

ANSWER

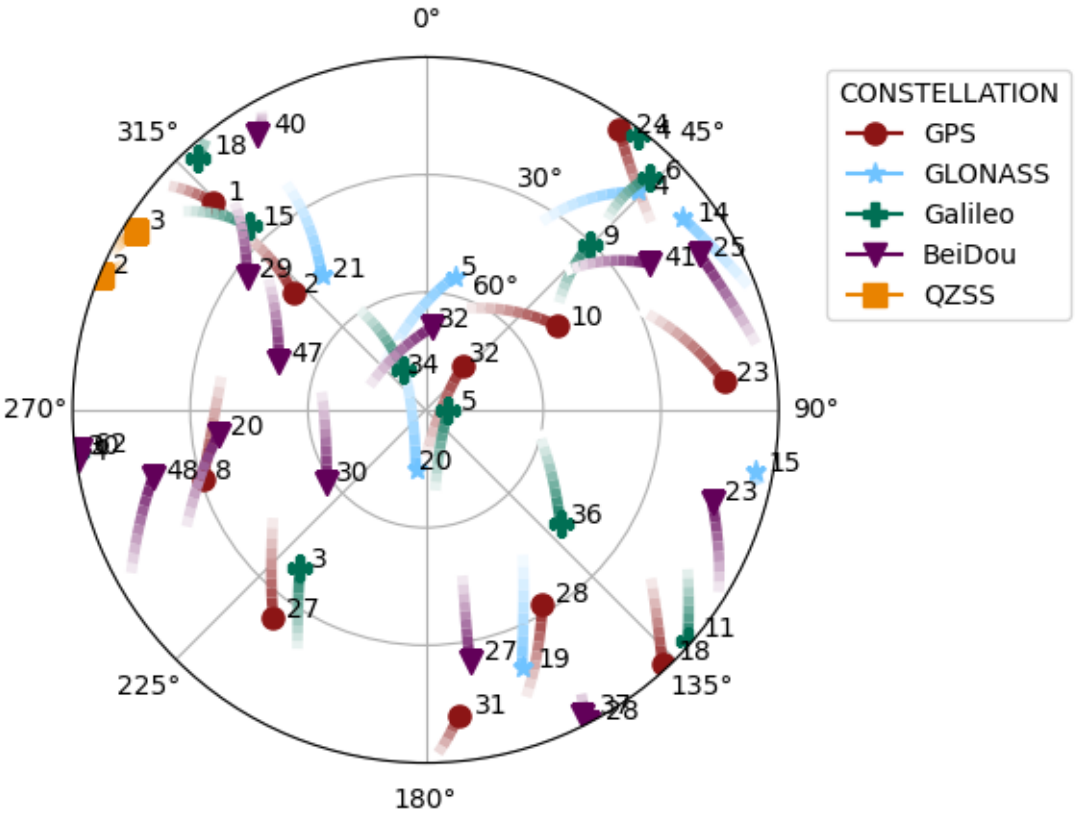
[27]:
```
slac_lat = 37 + 24/60 + 59.46807/3600
slac_lon = -122 - 12/60 - 15.36454/3600
slac_alt = 63.667 # m

slac_timestamp_start = datetime(2025, 9, 22, 9, tzinfo=ZoneInfo("America/
 ↪Los_Angeles"))
slac_timestamp_end = datetime(2025, 9, 22, 10, tzinfo=ZoneInfo("America/
 ↪Los_Angeles"))
```

```
print(f"Start: {slac_timestamp_start}")
print(f"End:   {slac_timestamp_end}")
```

```
Start: 2025-09-22 09:00:00-07:00
End:   2025-09-22 10:00:00-07:00
```

[28]:
```
plot_gps(slac_lat, slac_lon, slac_alt, slac_timestamp_start, slac_timestamp_end)
```

```
ephemeris dates needed: [datetime.date(2025, 9, 22)]
using previously downloaded file:
 /content/data/ephemeris/sp3/GFZ0MGXRAP_20252650000_01D_05M_ORB.SP3
using previously downloaded file:
 /content/data/ephemeris/sp3/GFZ0MGXRAP_20252650000_01D_05M_ORB.SP3
```



2.3 [2 Points]

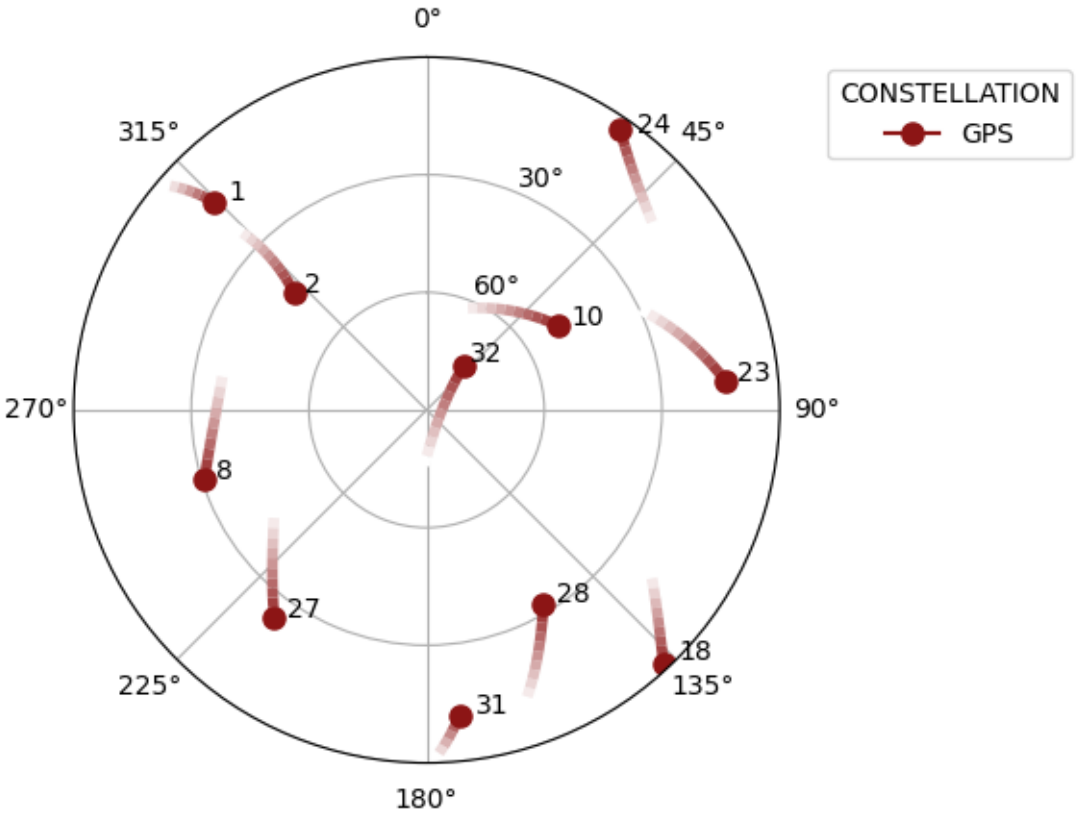
Repeat 2.2, but isolate only the GPS satellites.

Hint: Type `cropped_sp3` in a new cell and run the result. Which column of the data designates the satellite constellation? Also refer to the documentation: https://gnss-lib-py.readthedocs.io/en/latest/reference/navdata/navdata.html#navdata.NavData.where

ANSWER

[29]:
```
plot_gps(slac_lat, slac_lon, slac_alt, slac_timestamp_start, slac_timestamp_end,
         where_key_idx = "gnss_id",
         where_value = "gps")
```

```
ephemeris dates needed: [datetime.date(2025, 9, 22)]
using previously downloaded file:
 /content/data/ephemeris/sp3/GFZ0MGXRAP_20252650000_01D_05M_ORB.SP3
using previously downloaded file:
 /content/data/ephemeris/sp3/GFZ0MGXRAP_20252650000_01D_05M_ORB.SP3
```



2.4 [8 points]

Produce the skyplots for a 24-hour segment (e.g., midnight to midnight) at the following locations and times. Use all the satellite constellations from IGS and use an altitude of zero, if needed. Then, comment on the differences between them and why these differences occur.

1. The North Pole on December 25, 2024 in UTC. Please use true north (i.e., latitude of 90° and longitude of 0°) rather than magnetic north.
2. Singapore on March 20, 2025 (i.e., the vernal equinox Singapore Time)

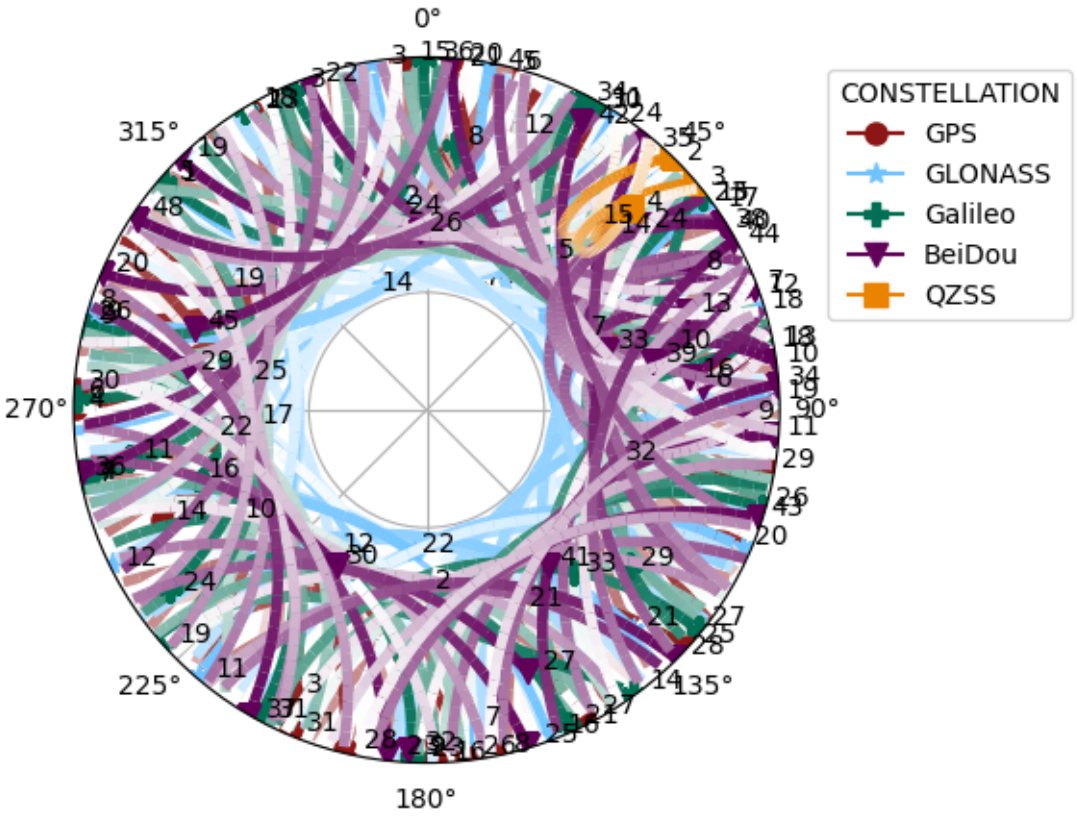
3. Tokyo on July 23, 2021 (i.e., the opening ceremony day for the 2020 Summer Olympics (it was held in 2021))
4. Rio de Janeiro on August 5, 2016 (i.e., the opening ceremony day for the 2016 Summer Olympics)

Just for fun, there is an interesting connection between the QZSS orbits and the sun. Check out the Analemma Wikipedia page for more.

ANSWER

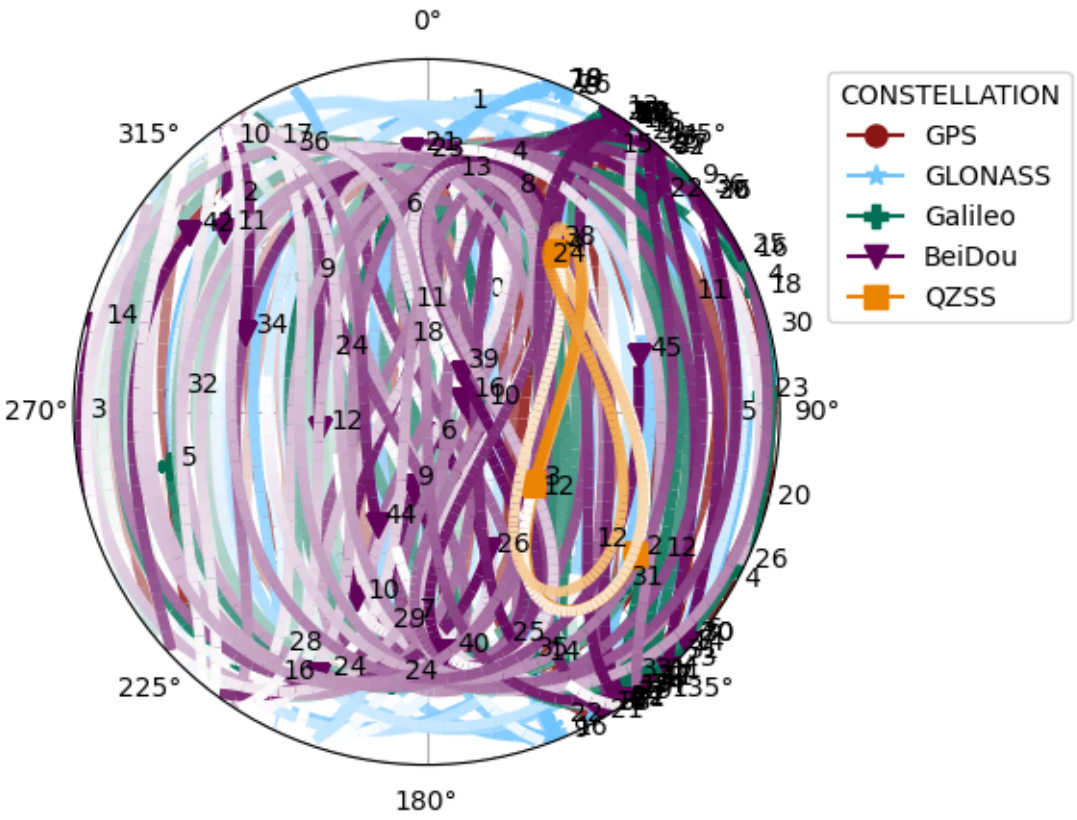
[30]:
```
plot_gps(
    lat = 90,
    lon = 0,
    alt = 0,
    timestamp_start = datetime(2024, 12, 25, tzinfo=timezone.utc),
    timestamp_end = datetime(2024, 12, 26, tzinfo=timezone.utc))
print("North Pole on December 25, 2024")
```

```
ephemeris dates needed: [datetime.date(2024, 12, 24), datetime.date(2024, 12,
25), datetime.date(2024, 12, 26)]
using previously downloaded file:
 /content/data/ephemeris/sp3/COD0MGXFIN_20243590000_01D_05M_ORB.SP3
using previously downloaded file:
 /content/data/ephemeris/sp3/COD0MGXFIN_20243600000_01D_05M_ORB.SP3
using previously downloaded file:
 /content/data/ephemeris/sp3/COD0MGXFIN_20243610000_01D_05M_ORB.SP3
using previously downloaded file:
 /content/data/ephemeris/sp3/COD0MGXFIN_20243590000_01D_05M_ORB.SP3
using previously downloaded file:
 /content/data/ephemeris/sp3/COD0MGXFIN_20243600000_01D_05M_ORB.SP3
using previously downloaded file:
 /content/data/ephemeris/sp3/COD0MGXFIN_20243610000_01D_05M_ORB.SP3
North Pole on December 25, 2024
```



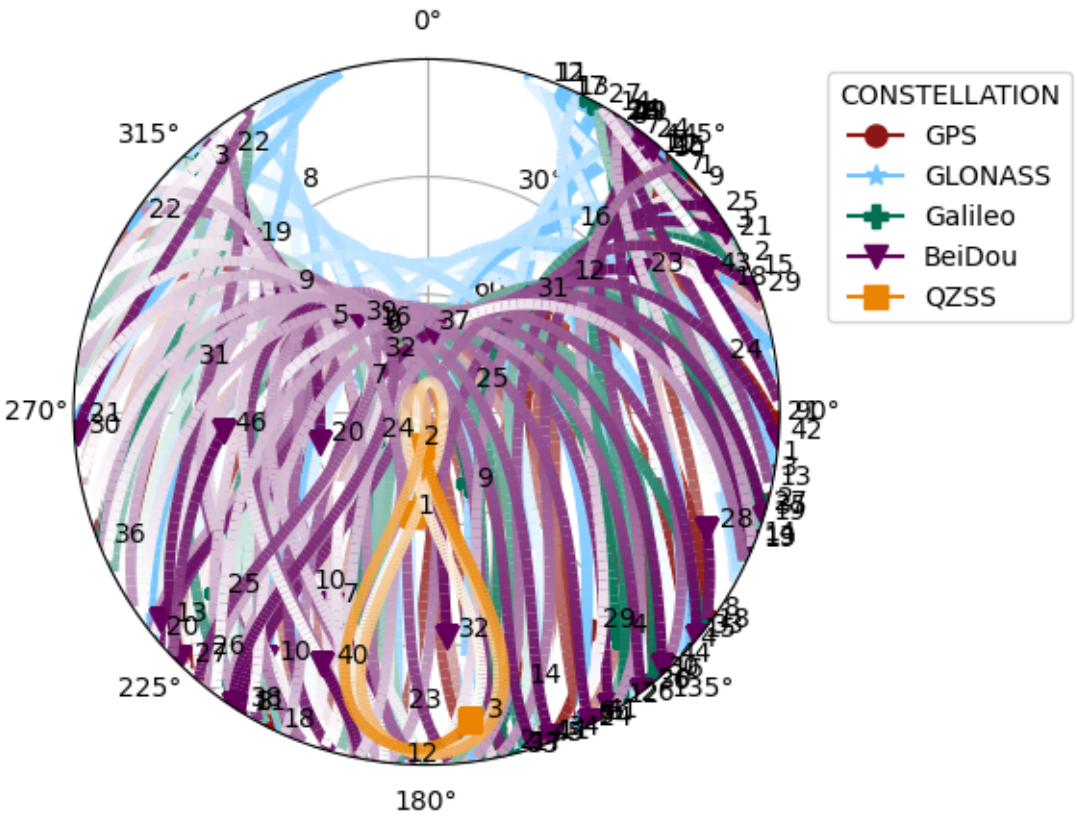
[31]:
```
plot_gps(
    lat = 1.3521,
    lon = 103.8198,
    alt = 0,
    timestamp_start = datetime(2025, 3, 20, tzinfo=ZoneInfo("Asia/Singapore")),
    timestamp_end = datetime(2025, 3, 21, tzinfo=ZoneInfo("Asia/Singapore")))
print("Singapore on March 20, 2025")
```

```
ephemeris dates needed: [datetime.date(2025, 3, 19), datetime.date(2025, 3, 20)]
using previously downloaded file:
 /content/data/ephemeris/sp3/CODOMGXFIN_20250780000_01D_05M_ORB.SP3
using previously downloaded file:
 /content/data/ephemeris/sp3/CODOMGXFIN_20250790000_01D_05M_ORB.SP3
using previously downloaded file:
 /content/data/ephemeris/sp3/CODOMGXFIN_20250780000_01D_05M_ORB.SP3
using previously downloaded file:
 /content/data/ephemeris/sp3/CODOMGXFIN_20250790000_01D_05M_ORB.SP3
Singapore on March 20, 2025
```



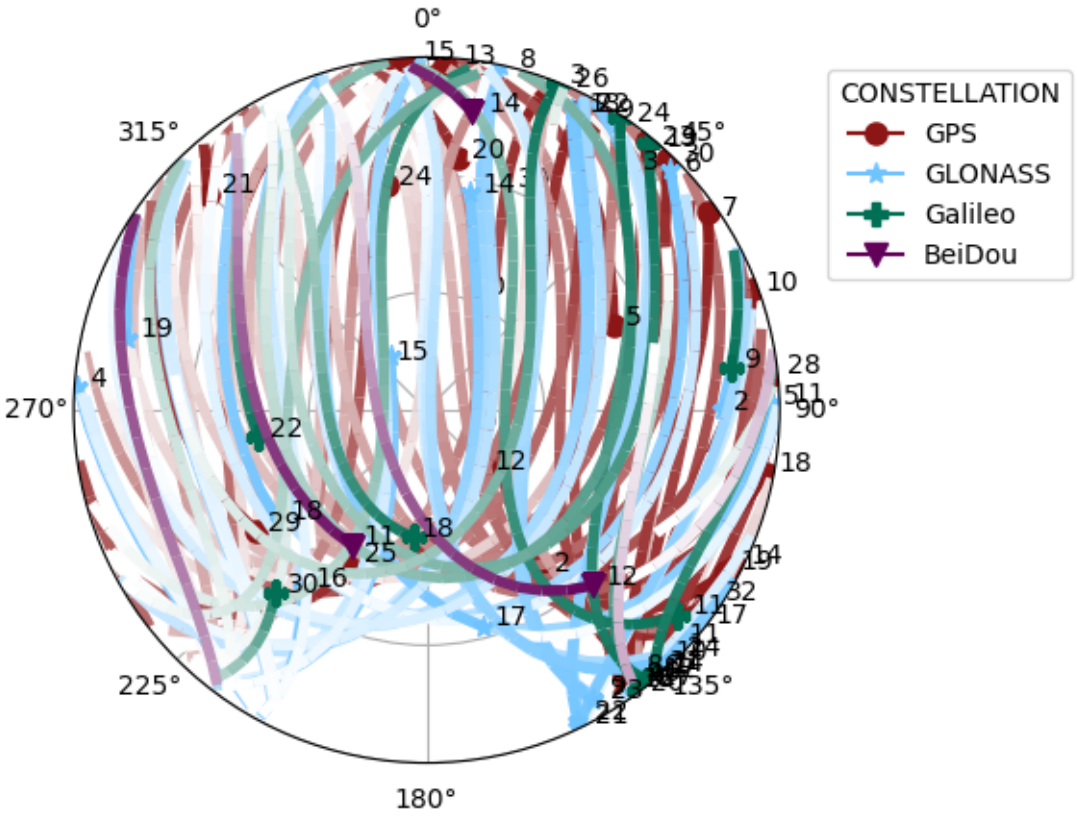
[32]:
```
plot_gps(
    lat = 35.6764,
    lon = 139.6500,
    alt = 0,
    timestamp_start = datetime(2021, 7, 23, tzinfo=ZoneInfo("Asia/Tokyo")),
    timestamp_end = datetime(2021, 7, 24, tzinfo=ZoneInfo("Asia/Tokyo")))
print("Tokyo on July 23, 2021")
```

```
ephemeris dates needed: [datetime.date(2021, 7, 22), datetime.date(2021, 7, 23)]
using previously downloaded file:
 /content/data/ephemeris/sp3/CODOMGXFIN_20212030000_01D_05M_ORB.SP3
using previously downloaded file:
 /content/data/ephemeris/sp3/CODOMGXFIN_20212040000_01D_05M_ORB.SP3
using previously downloaded file:
 /content/data/ephemeris/sp3/CODOMGXFIN_20212030000_01D_05M_ORB.SP3
using previously downloaded file:
 /content/data/ephemeris/sp3/CODOMGXFIN_20212040000_01D_05M_ORB.SP3
Tokyo on July 23, 2021
```



[33]:
```
plot_gps(
    lat = -22.9068,
    lon = -43.1729,
    alt = 0,
    timestamp_start = datetime(2016, 8, 5, tzinfo=ZoneInfo("Brazil/East")),
    timestamp_end = datetime(2016, 8, 6, tzinfo=ZoneInfo("Brazil/East")))
print("Rio de Janeiro on August 5, 2016")
```

```
ephemeris dates needed: [datetime.date(2016, 8, 5), datetime.date(2016, 8, 6)]
using previously downloaded file:
 /content/data/ephemeris/sp3/com19085.sp3
using previously downloaded file:
 /content/data/ephemeris/sp3/com19086.sp3
using previously downloaded file:
 /content/data/ephemeris/sp3/com19085.sp3
using previously downloaded file:
 /content/data/ephemeris/sp3/com19086.sp3
Rio de Janeiro on August 5, 2016
```


0°
45°
90°
135°
180°
225°
270°
315°
CONSTELLATION
GPS
GLONASS
Galileo
BeiDou

Problem 3: GPS User Segment [20 points + 2 bonus]

Goal of this problem: Students will learn about the GPS user segment and the key geometric interpretation of GPS navigation.

Consider a system of 2D buzzers in a 2D coordinate plane, each equipped with an atomic clock, where all buzzer clocks are perfectly synchronized within the system.

Each buzzer broadcasts: 1. a buzzing sound, 2. the buzzing sound's corresponding precise time of transmission $t^{(k)}$, and 3. the buzzer's number k.

Additionally consider that you have a receiver, that is equipped with a local clock that records the time of reception $t_u^{(k)}$ for each buzzing sound. However, the receiver clock has an unknown clock bias t_b, which means the true reception time for the sound from buzzer k is $t_u^{(k)} - t_b$.

For this problem, we will build a 2D trilateration system based on this system of buzzers. Assume that the speed of sound is $c_{\text{sound}} = 340$ m/s. Assume your position is stationary, and the receiver clock bias is constant for all buzzer sounds you receive.

Instruction for Plots: For your plot, please make sure the following are satisfied. * Ensure that x is along the horizontal axis and y is along the vertical axis. * Ensure your axes are labeled and the **aspect ratio is equal**. * **Include grid lines and a legend** that distinguishes the position of buzzers from the possible positions. * **Please use the same plot bounds for all parts of this problem**: [-3000, 3000] for x and [-3000, 3000] for y.

It may be helpful to leverage the navigation equations we learned in Lecture 3 for this problem.

3.1 [2 points]

You know that Buzzer 1 is located at (500 m, 1000 m) in the global frame of reference.

Using symbols $t^{(1)}$, t_b, and $t_u^{(1)}$, write the equation satisfied by your true position (x, y), when you receive the measurement from Buzzer 1.

ANSWER

$$\vec{x}^{(1)} - \vec{x} = \begin{pmatrix} x \\ y \end{pmatrix} - \begin{pmatrix} 500 \\ 1000 \end{pmatrix} = c(t_u^{(1)} - t_b - t^{(1)})$$

$$\begin{pmatrix} x \\ y \end{pmatrix} = c(t_u^{(1)} - t_b - t^{(1)}) + \begin{pmatrix} 500 \\ 1000 \end{pmatrix}$$

3.2 [3 points]

Assume $t^{(1)} = 0.5$ s, $t_b = +0.1$ s, and $t_u^{(1)} = 4.1$ s.

Plot the geometry in the xy plane that satisfies your equation from part 3.1, along with the location of Buzzer 1.

Hint: You might find it easier to graph the geometry if you first create the array of 100 evenly spaced (angle) points going from 0 to 2π radians.

ANSWER

[71]:
```
import numpy as np
import plotly.graph_objs as px_go
from dataclasses import dataclass
import matplotlib.pyplot as plt
```

[72]:
```
c_speed_sound = 340 # m/s
t_1 = 0.5 # s
t_b = 0.1 # s
t_u_1 = 4.1 # s
```

[73]:
```
@dataclass
class Buzzer:
  x: float
  y: float
  dist_lower: float
  dist_upper: float

  inner_x_points: np.array
  inner_y_points: np.array
  outer_x_points: np.array
  outer_y_points: np.array
```

[74]:
```
def get_buzzer(buzzer_x, buzzer_y, t_k, t_b, t_u_k):

  dist_lower = c_speed_sound * (t_u_k - t_b - t_k)
  dist_upper = c_speed_sound * (t_u_k + t_b - t_k)

  circle_points = np.linspace(0, 2*np.pi, 1000)

  inner_x_points = np.cos(circle_points) * dist_lower + buzzer_x
  inner_y_points = np.sin(circle_points) * dist_lower + buzzer_y
  outer_x_points = np.cos(circle_points) * dist_upper + buzzer_x
  outer_y_points = np.sin(circle_points) * dist_upper + buzzer_y

  return Buzzer(
      x=buzzer_x,
      y=buzzer_y,
      dist_lower=dist_lower,
      dist_upper=dist_upper,
      inner_x_points=inner_x_points,
      inner_y_points=inner_y_points,
      outer_x_points=outer_x_points,
      outer_y_points=outer_y_points)
```

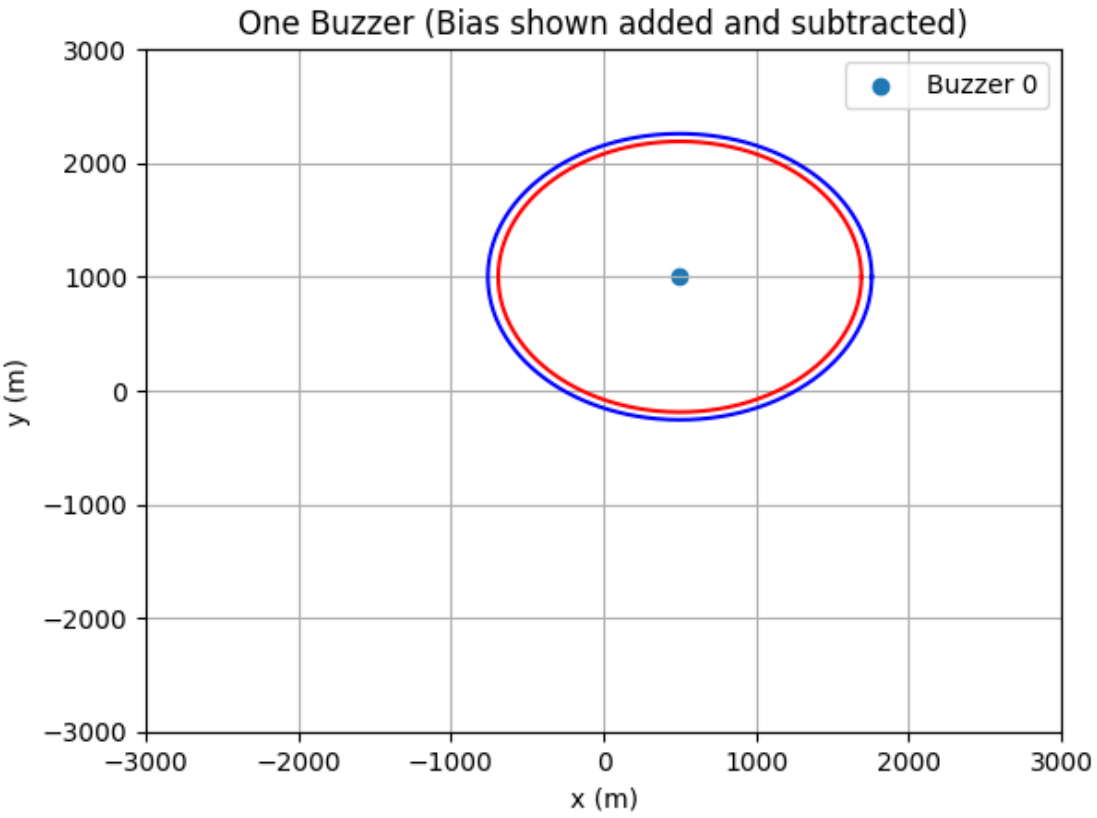
[75]:
```
def plot_buzzers(buzzers: list[Buzzer], plot_title: str, range_type = "both",␣
 ↪use_plotly = False):
  plot_objects = []
```

```
if isinstance(buzzers, Buzzer):
  buzzers = [buzzers]

if use_plotly:
  for i, buzzer in enumerate(buzzers):
    plot_objects += [
      px_go.Scatter(
        name=f'Buzzer {i}',
        x=[buzzer.x],
        y=[buzzer.y],
        mode='markers',
        marker=dict(color='red', size=10),
        showlegend=True
      ),
      px_go.Scatter(
          # name=f'Buzzer {i} Inner Estimate',
          x=buzzer.inner_x_points,
          y=buzzer.inner_y_points,
          mode='markers',
          marker=dict(color='red', size=2),
          showlegend=True
      ),
        px_go.Scatter(
          # name=f'Buzzer {i} Outer Estimate',
          x=buzzer.outer_x_points,
          y=buzzer.outer_y_points,
          mode='markers',
          marker=dict(color='blue', size=2),
          showlegend=True
      )
    ]

  fig = px_go.Figure(plot_objects)
  fig.update_layout(
    xaxis_title='x (m)',
    yaxis_title='y (m)',
    xaxis_range=[-3000, 3000],
    yaxis_range=[-3000, 3000],
    title=plot_title,
    hovermode="x"
  )
  fig.show()

else:
```

```
    for i, buzzer in enumerate(buzzers):
      plt.scatter(buzzer.x, buzzer.y, label=f"Buzzer {i}")
      if range_type in ["both", "inner"]:
        plt.plot(buzzer.inner_x_points, buzzer.inner_y_points, 'r')
      if range_type in ["both", "outer"]:
        plt.plot(buzzer.outer_x_points, buzzer.outer_y_points, 'b')

    plt.legend()
    plt.title(plot_title)
    plt.xlim(-3000, 3000)
    plt.ylim(-3000, 3000)
    plt.xlabel("x (m)")
    plt.ylabel("y (m)")
    plt.grid(True)
    plt.show()
```

[76]:
```
buzzer_1 = get_buzzer(500, 1000, t_1, t_b, t_u_1)
```

[77]:
```
plot_buzzers(buzzer_1, "One Buzzer (Bias shown added and subtracted)")
```



[77]:

3.3 [2 points]

In addition to Buzzer 1, your receiver also obtains a sound from Buzzer 2, which is located at (-1000 m, 0 m).

Using symbols $t^{(1)}$, $t^{(2)}$, t_b, $t_u^{(1)}$, and $t_u^{(2)}$, write the two equations satisfied by your true position (x,y).

ANSWER

$$\begin{pmatrix} x \\ y \end{pmatrix} = c(t_u^{(1)} - t_b - t^{(1)}) + \begin{pmatrix} 500 \\ 1000 \end{pmatrix}$$

$$\begin{pmatrix} x \\ y \end{pmatrix} = c(t_u^{(2)} - t_b - t^{(2)}) + \begin{pmatrix} -1000 \\ 0 \end{pmatrix}$$

3.4 [2 points]

Assume $t^{(1)} = 0.5$s, $t^{(2)}$ =1.5 s, $t_b = +0.1$ s, $t_u^{(1)} = 4.1$ s, and $t_u^{(2)} = 4.8$ s.

Plot the two geometries that satisfy each equation, along with the location of the 2 buzzers.

From the plot, how many possible locations exist as the user position?

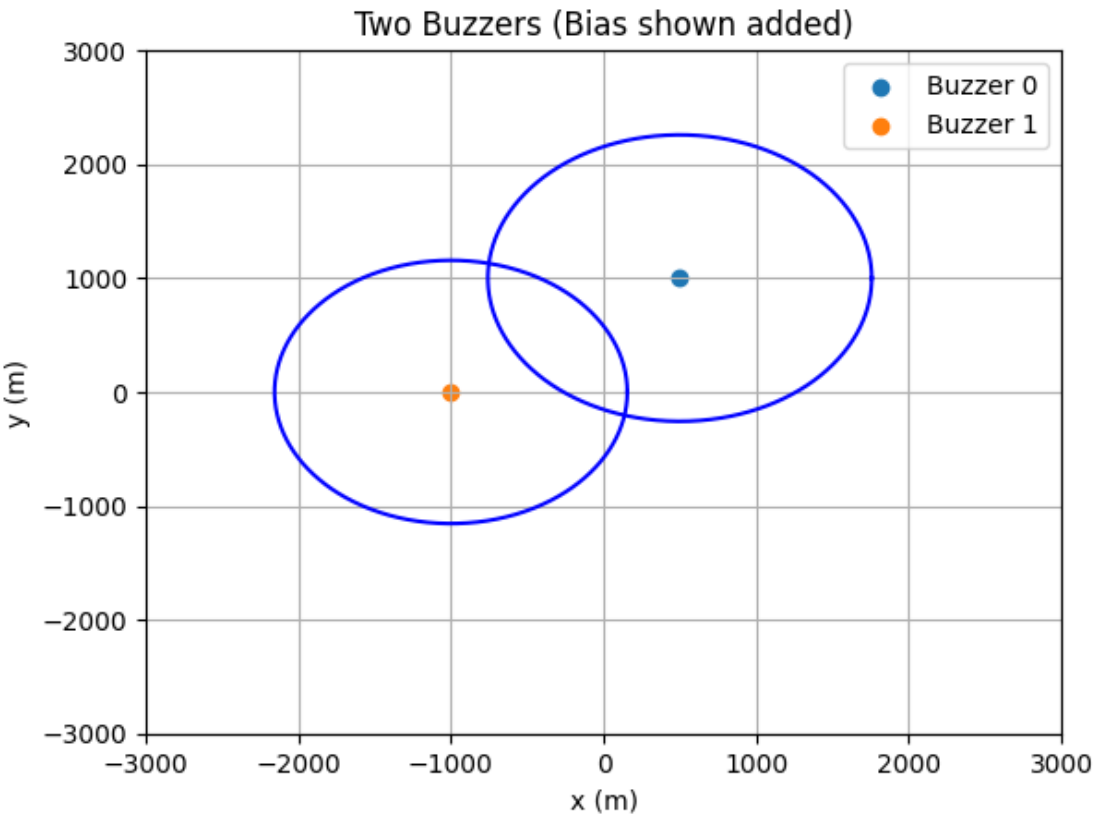
You do not have to compute the positions.

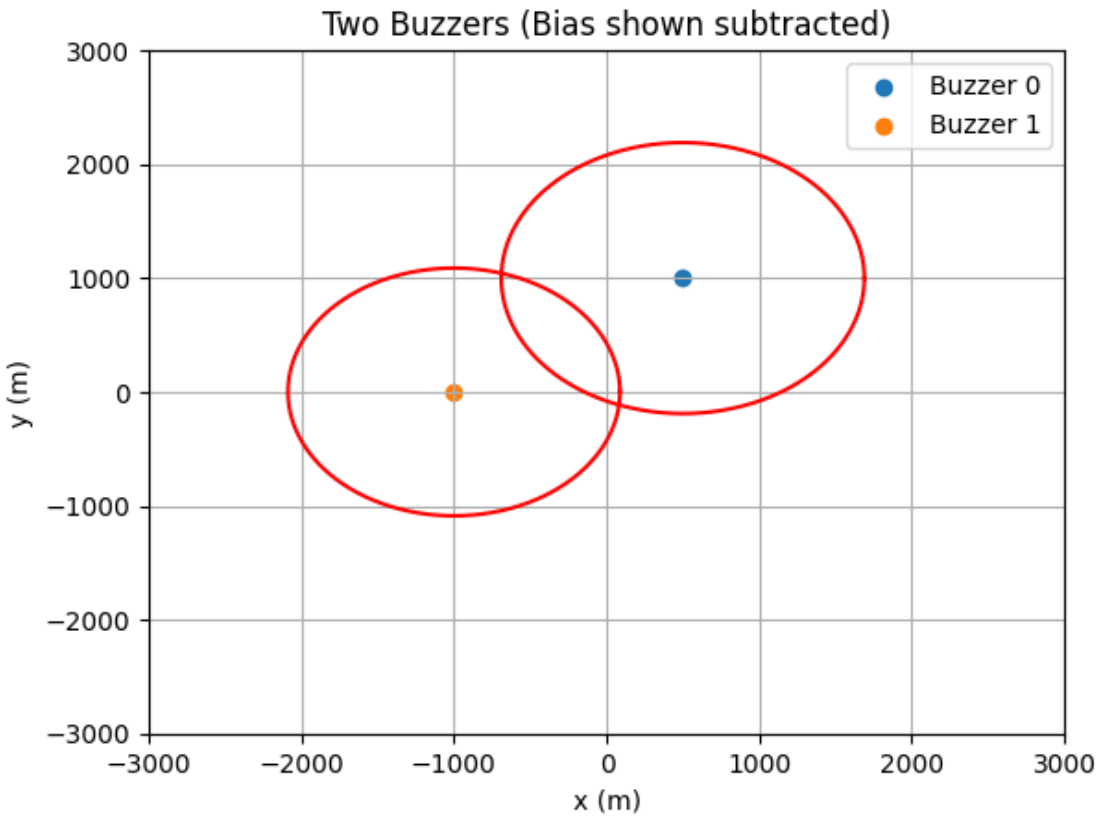
ANSWER

[78]:
```
t_2 = 1.5 # s
t_u_2 = 4.8 # s
```

[79]:
```
buzzer_2 = get_buzzer(-1000, 0, t_2, t_b, t_u_2)
```

[80]:
```
plot_buzzers([buzzer_1, buzzer_2], "Two Buzzers (Bias shown added)",␣
 ↪range_type="outer")
plot_buzzers([buzzer_1, buzzer_2], "Two Buzzers (Bias shown subtracted)",␣
 ↪range_type="inner")
```


Two Buzzers (Bias shown added)
Buzzer 0
Buzzer 1
3000
2000
1000
0
−1000
−2000
−3000
y (m)
−3000
−2000
−1000
0
1000
2000
3000
x (m)



3.5 [2 points]

In addition to Buzzer 1 and 2, your receiver also obtains a sound from Buzzer 3, which is located at (+1000 m, -1000 m). Using symbols $t^{(1)}$, $t^{(2)}$, $t^{(3)}$, t_b, $t_u^{|(1)}$, $t_u^{(2)}$, and $t_u^{(3)}$, write the three equations satisfied by your true position (x, y).

ANSWER

$$\begin{pmatrix} x \\ y \end{pmatrix} = c(t_u^{(1)} - t_b - t^{(1)}) + \begin{pmatrix} 500 \\ 1000 \end{pmatrix}$$

$$\begin{pmatrix} x \\ y \end{pmatrix} = c(t_u^{(2)} - t_b - t^{(2)}) + \begin{pmatrix} -1000 \\ 0 \end{pmatrix}$$

$$\begin{pmatrix} x \\ y \end{pmatrix} = c(t_u^{(3)} - t_b - t^{(3)}) + \begin{pmatrix} -1000 \\ 1000 \end{pmatrix}$$

3.6 [2 points]

Assume $t^{(1)}$ =0.5 s, $t^{(2)}$ =1.5 s, $t^{(3)}$ =0.0 s, t_b = +0.1 s, $t_u^{(1)}$ = 4.1 s, $t_u^{(2)}$ = 4.8 s, and $t_u^{(3)}$ = 5.2 s. Plot the three geometries that satisfy each equation, along with the location of the 3 buzzers. Are

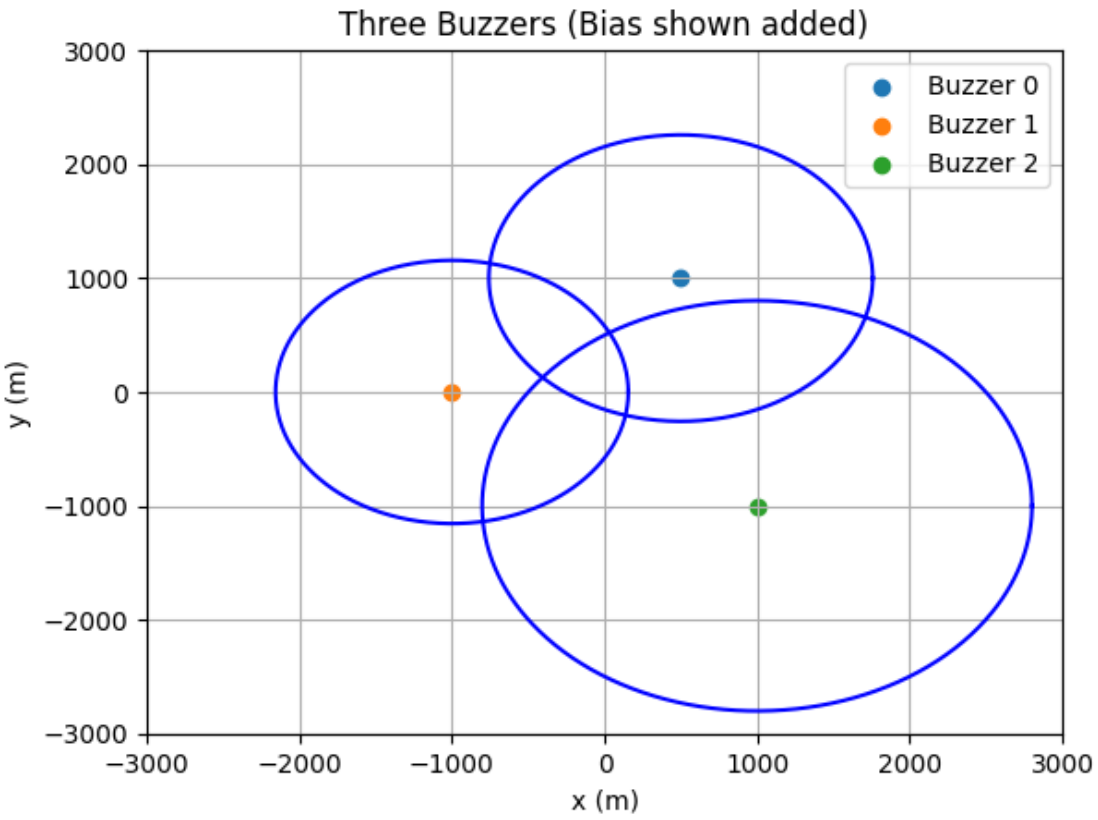
there any locations that satisfies the three equations simultaneously?

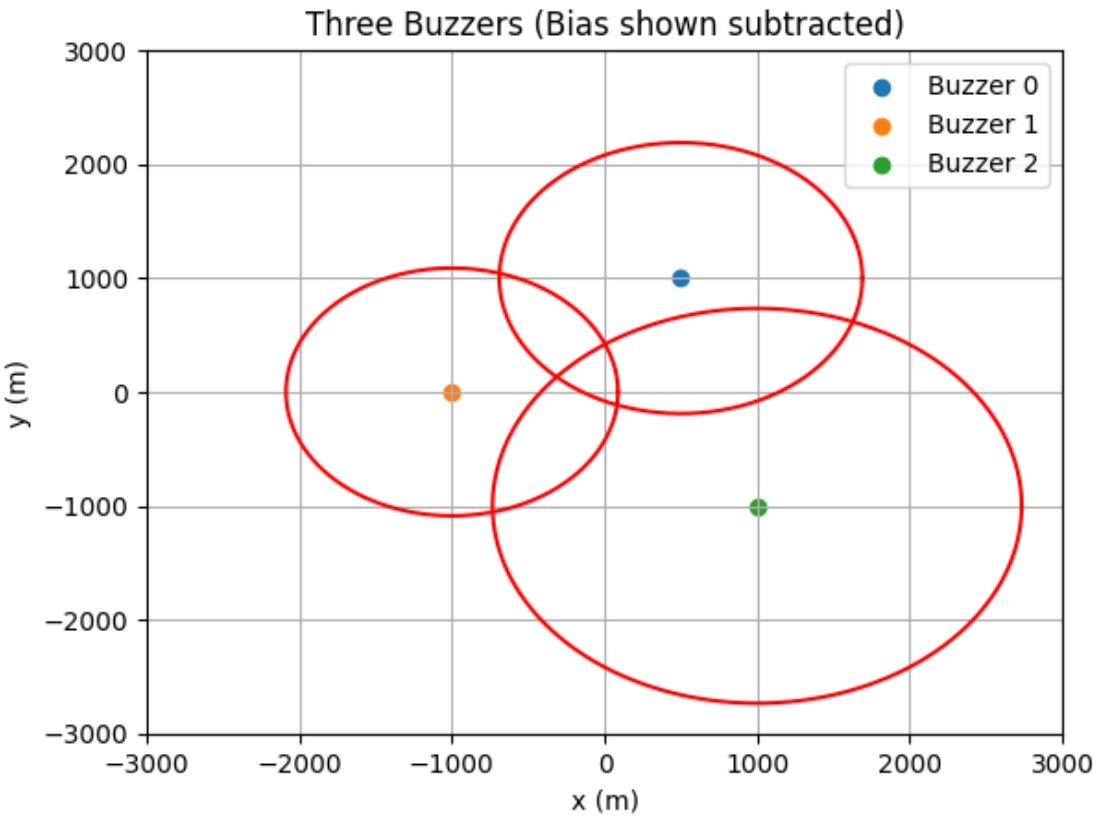
ANSWER

[81]:
```
t_3 = 0 # s
t_u_3 = 5.2 # s
```

[82]:
```
buzzer_3 = get_buzzer(1000, -1000, t_3, t_b, t_u_3)
```

[83]:
```
plot_buzzers([buzzer_1, buzzer_2, buzzer_3], "Three Buzzers (Bias shown␣
↪added)", range_type="outer")
plot_buzzers([buzzer_1, buzzer_2, buzzer_3], "Three Buzzers (Bias shown␣
↪subtracted)", range_type="inner")
```





3.7 [3 points]

Based on the result you got in problem 3.6, do you think the true clock bias t_b is greater than, smaller than, or equivalent to t_b =0.1 s? Explain the reason.

Assume other parameters are fixed and correct ($t^{(1)}$ =0.5 s, $t^{(2)}$ =1.5 s, $t^{(3)}$ =0.0 s, $t_u^{(1)}$ = 4.1 s, $t_u^{(2)}$ = 4.8 s, and $t_u^{(3)}$ = 5.2 s)

ANSWER

The clock bias is likely larger than 0.1 seconds because the three inner and outer bounds of the pseudorange for the satellites do not intersect at all. A larger bias will mean more "area" in which the regions will be able to intersect.

3.8 [4 points]

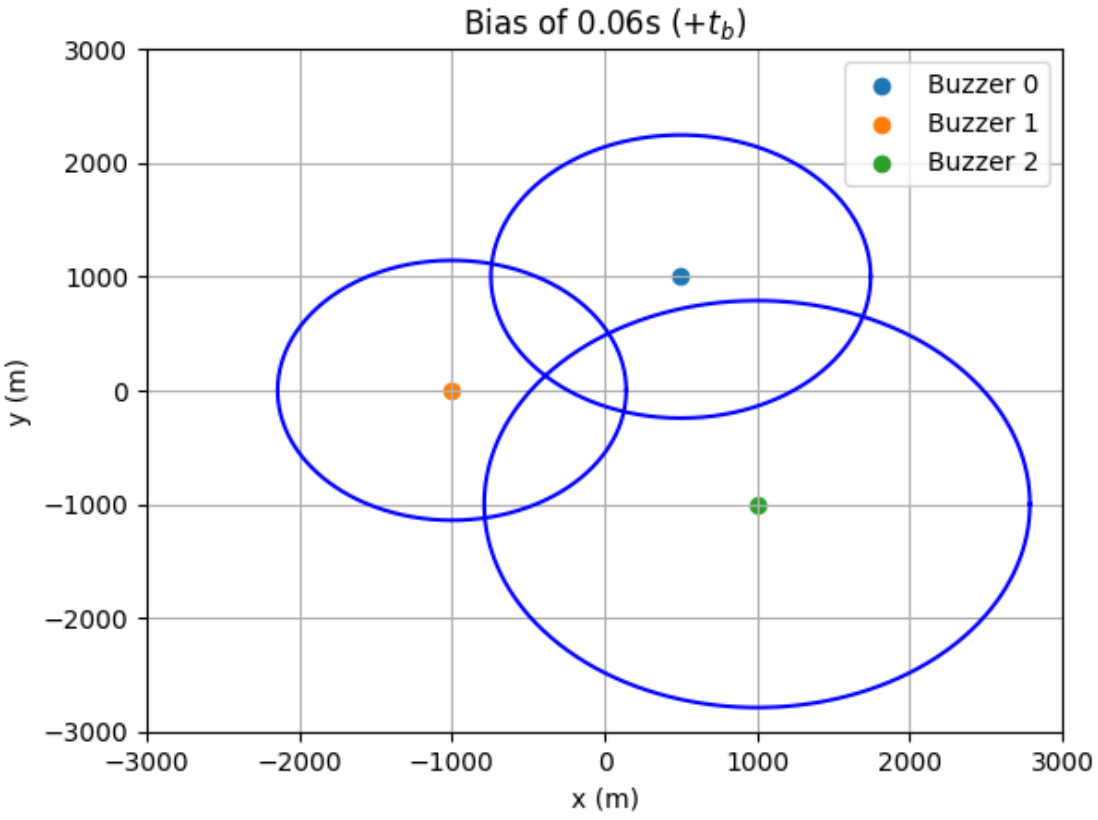
For three different clock biases: t_b = +0.06 s, t_b =+0.56 s, and t_b =+1.06 s, plot the three geometries that satisfy each equation of the buzzer measurements, along with the location of the 3 buzzers. Assume other parameters are fixed and correct ($t^{(1)}$ =0.5 s, $t^{(2)}$ =1.5 s, $t^{(3)}$ =0.0 s, $t_u^{(1)}$ = 4.1 s, $t_u^{(2)}$ = 4.8 s, and $t_u^{(3)}$ = 5.2 s).

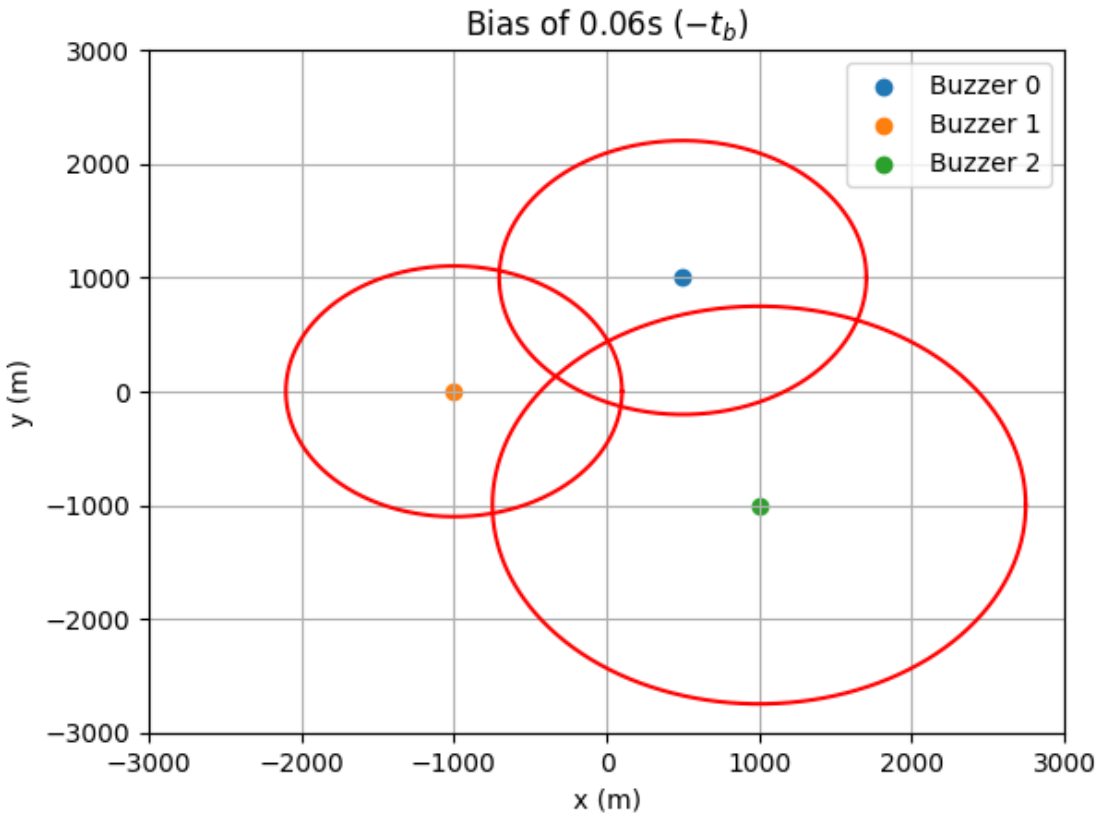
Which clock bias do you think is closest to the correct clock bias? Does the result match with your answer at problem 3.7?

ANSWER

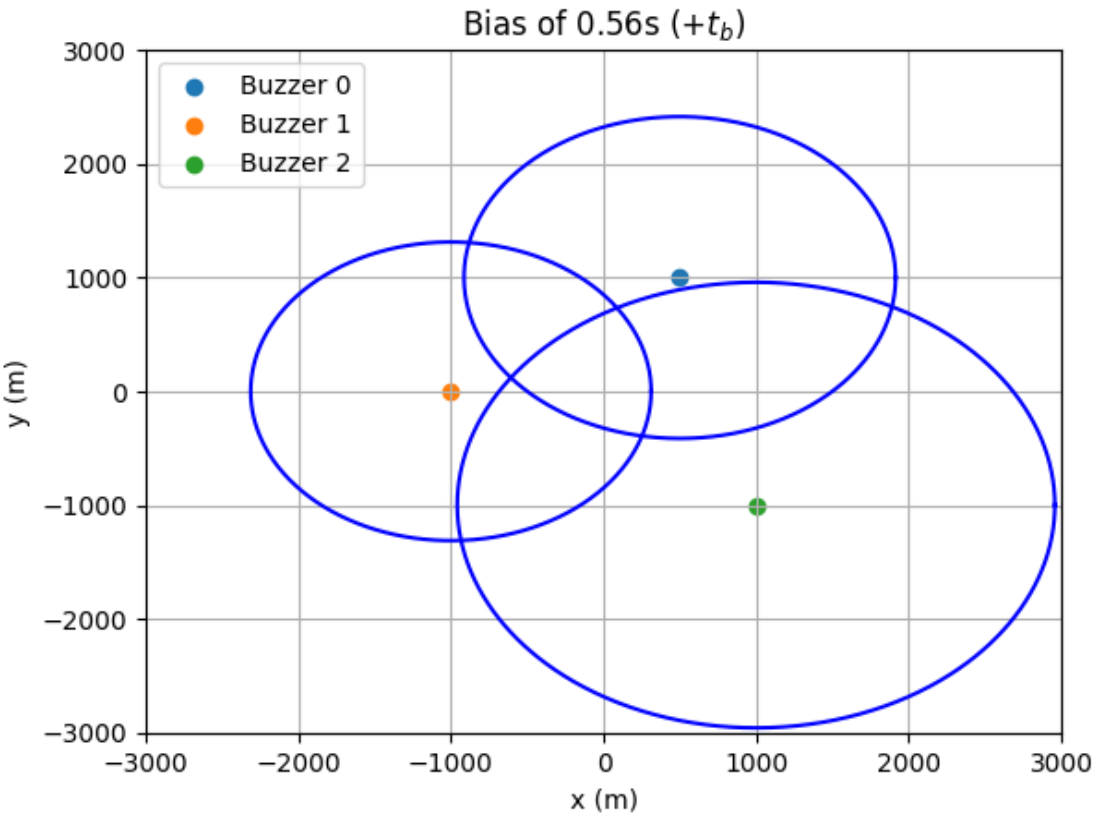
[84]:
```
def plot_with_bias(bias):

  buzzer_1 = get_buzzer(500, 1000, t_1, bias, t_u_1)
  buzzer_2 = get_buzzer(-1000, 0, t_2, bias, t_u_2)
  buzzer_3 = get_buzzer(1000, -1000, t_3, bias, t_u_3)

  plot_buzzers([buzzer_1, buzzer_2, buzzer_3], f"Bias of {bias}s ($+t_b$)",␣
↪"outer")
  plot_buzzers([buzzer_1, buzzer_2, buzzer_3], f"Bias of {bias}s ($-t_b$)",␣
↪"inner")
```

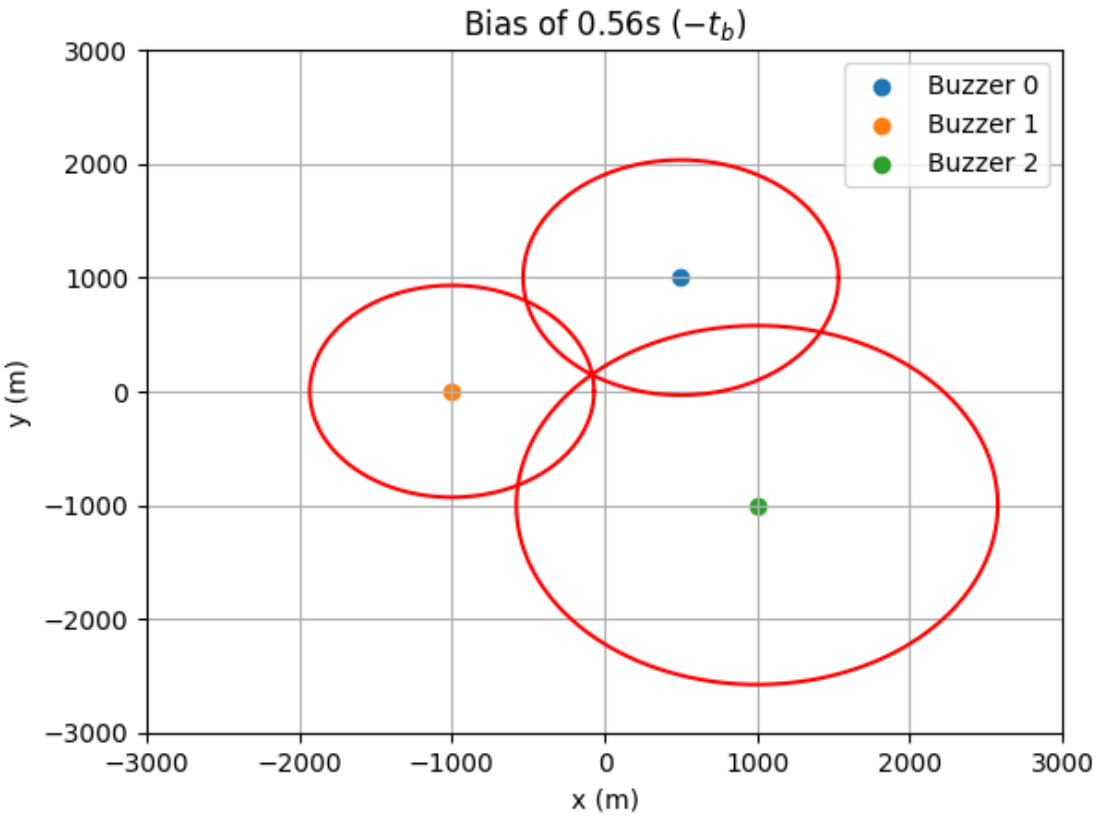
[85]:
```
plot_with_bias(0.06)
```



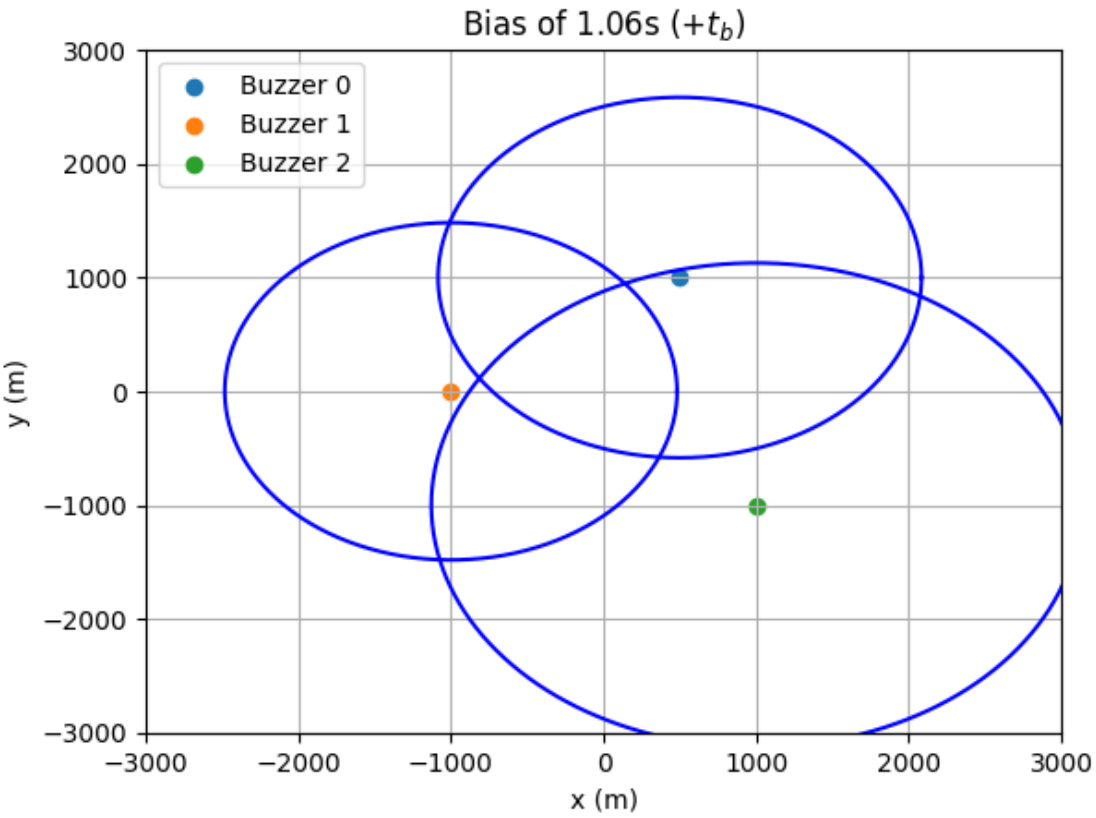


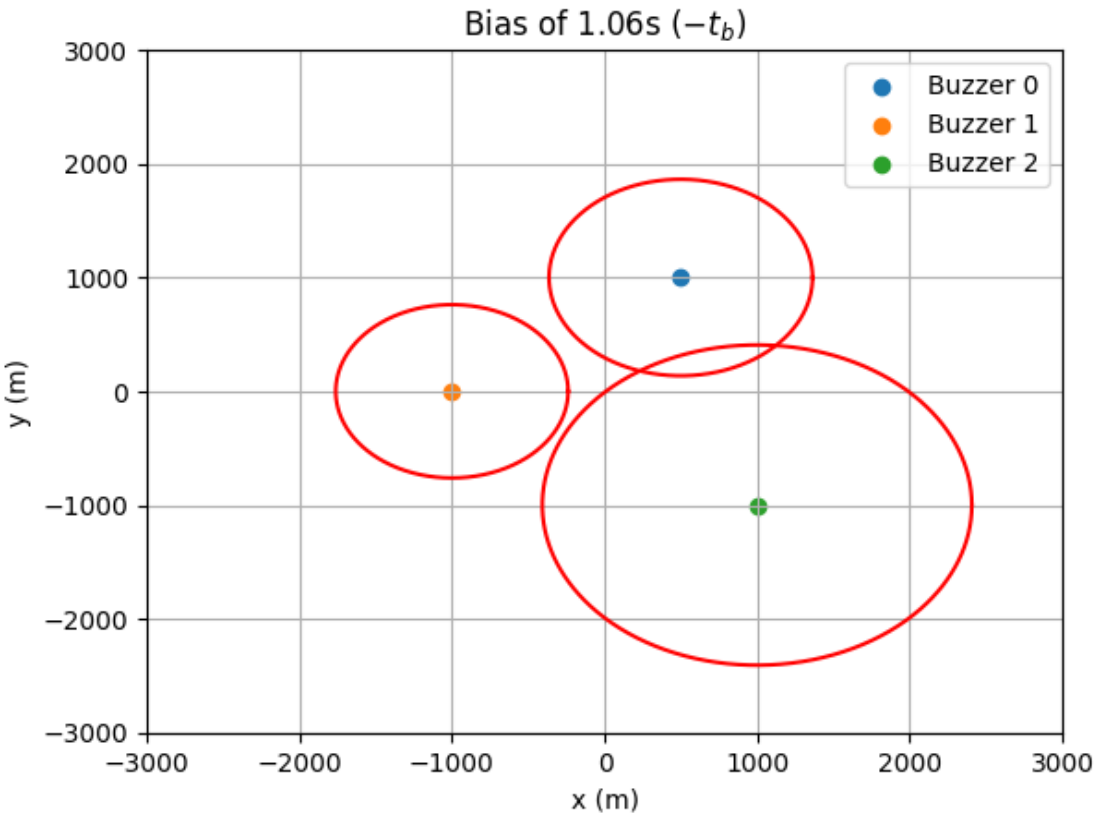
[86]:
```
plot_with_bias(0.56)
```


Bias of 0.56s (+t_b)
Buzzer 0
Buzzer 1
Buzzer 2
3000
2000
1000
0
−1000
−2000
−3000
y (m)
−3000
−2000
−1000
0
1000
2000
3000
x (m)



[87]:
```
plot_with_bias(1.06)
```


Bias of 1.06s ($+t_b$)
Buzzer 0
Buzzer 1
Buzzer 2
3000
2000
1000
0
−1000
−2000
−3000
y (m)
−3000
−2000
−1000
0
1000
2000
3000
x (m)



I think (or would like to think) 0.56 is the likely clock bias because when the bias is subtracted (as signified by the plot in red), the 3 buzzers just barely intersect.

Note You will learn how to solve these set of equations (for the 3d + clock bias case) in Lecture 3.

3.9 Bonus [2 points]

Suppose you have a fourth buzzer. However, this fourth buzzers are equipped with cheaper clocks, and sometimes it sends out a biased transmission time.

Using the measurement from the four buzzers, can you think of a way to tell if the forth buzzer is sending out the correct transmission time?

You can assume the other three buzzers are sending out the correct transmission time, and we know the true exact position of the fourth buzzer.

ANSWER

You would solve the problem with the previous 3 buzzers, then get the pseudorange of the 4th buzzer, and if it intersects at the same point, then the 4th buzzer clock is correct.

Problem 4: PNT System Requirements [8 points]

Goal of this problem: Students will learn to think critically about the positioning, navigation, and timing requirements for a breadth of systems.

Below is a list of jargon that we will discuss later in this course, but which may be useful for this problem:

1. **Position**: Where the receiver is in reference to a specific coordinate system.
2. **Navigation**: The receiver's traversal through a coordinate system.
3. **Timing**: What time the receiver clock reads relative to a reference time system.
4. **PNT**: Position, Navigation, and Time
   - https://scpnt.stanford.edu/
5. **Accuracy**: How close is the estimated position/time to the true position/time? Often, this is based on the root-mean-squared error or the standard deviation of the errors.
   - https://gssc.esa.int/navipedia/index.php?title=Accuracy
6. **Availability**: How often is a positioning or timing solution available?
   - https://gssc.esa.int/navipedia/index.php?title=Availability
7. **Integrity**: Integrity is a large subfield of navigation. However, we will not explore this too deeply right now. In a simplified sense, how often is the position/time estimate trustworthy, especially for safety-critical applications?
   - https://gssc.esa.int/navipedia/index.php/Integrity

In addition to the topics discussed in the first lecture, here is a list of example topics covered when students were describing their academic and research interests on Monday. We apologize in advance if you do not see your interest area on the list!

1. **Outdoor localization, robotics, and autonomy**: Localizing a robot in diverse environments (e.g., a farm, a desert, a city).
   - See also the webpage and research from Prof.Gao.
2. **Space weather and ionospheric plasma**: Assessing the severity and extent of space weather events (e.g., solar effects on the ionosphere).
   - See also the webpage and research from Prof.Elschot and Prof.Walter
3. **Formation flying satellites** Determining absolute and relative position of multiple satellites orbiting in close proximity or even conducting rendevous.
   - See also the webpage and research from Prof.D'Amico
4. **Lunar PNT** Determining position and time accurately on the Moon is crucial to returning humans to the Moon. To meet this need, there are plans to create another satellite navigation system for the Moon (e.g. LunaNet).
   - See also the webpage for research from the Stanford NAVlab
5. **Structural health monitoring** Assessing the movement and deformation of structures (e.g., buildings or plane wings) due to loading (e.g., earthquakes or wind gusts).
   - See also the webpage and research from Prof.Chang (not directly GPS)
6. **Time synchronization for powergrids** Using GPS signals for accurate timekeeping to maintain synchronized operations and manage power flows effectively.

- See here for more GPS timing technology applications.

7. **Aircraft navigation** Determining the aircraft trajectory to ensure a safe take-off or landing even in difficult conditions (e.g., rain, fog, snow).
   - See also the Flight24 aircraft tracker and research from Prof.Walter and Gao.
8. **Maritime navigation** Determining the ship trajectory to ensure an optimized travel time and to safely dock at a port. See also the MarineTraffic ship tracker and research from Prof.Walter and Gao.
   - Link 1
   - Link 2

Looking for more ideas:

- Chapters 1-3 of the GPS Textbook
- InsideGNSS
- GPS World

4.1 [4 points]

Select four topics that use GPS. These can be any of the topics above, in lecture, or any you devise yourself.

Then, complete the table below with the loosest system requirements that would still meet the objective. We have provided one example below.

GPS is a stealth utility, so please consider a diverse set of applications and think critically about each application.

Topic	Accuracy	Availability	Integrity
Tectonic Plate Motion	*Millimeter (Position)*	*Monthly*	*Not safety critical*

ANSWER

Topic	Accuracy	Availability	Integrity
Satellite Orbits	*Millimeter (Position)*	*Quick (~10-100Hz)*	*Safety critical (for 100% GPS reliant satellites and under certain conditions)*
(Personal) Object Location Tracking	*Meter (Position)*	*Somewhat frequent (second to minute)*	*Not safety critical, but definitely useful*
Stock Market	*Microsecond? or Millisecond (Time)*	*Very very frequent*	*Financially critical*
Outdoor Localization	*Millimeter (Position)*	*Frequent (millisecond)*	*Not extremely critical*

4.2 [4 points]

For each topic, defend your requirements as a system engineer. Why is this the loosest you can tolerate?

Example:

- According to NOAA, plate tectonics move at an average rate of roughly 1.5~cm a year (NOAA Website). We need sub-centimeter accuracy to see changes over the course of a year, especially in regions that are less tectonically active.
- Although faster update rates would be nice, monthly data is likely sufficient to see trends across the year and between years without being too sensitive to seasonal variability.
- Tectonic plate motion is not a safety-critical application, so the integrity requirements are likely not too strict since we can handle outliers in post-processing.
  - Note however that earthquake early warning systems would be a safety-critical application and the integrity requirements would be more important.

ANSWER

Satellites

- A LEO satellite travels at about $7.8km/s$, so having GPS at a $100Hz$ ($10ms$) rate means it travels about $7.8m$ every cycle.
- If the GPS measurement is off by even $1mm$, it means a predicted angle of $0.007rad$. While this may not seem like much, propogating this orbit means a maximum of about $817m$.
- For orbital collision, this is not ideal. For a $0.1m$ deviation, this means over $8km$ of difference. Although the bounds of this trajectory can be narrowed as time progresses, it is not ideal to have such large uncertainties for multi-million dollar equipment in orbit.
- Realistically, satellites do not rely solely on GPS for their navigation, but this assumes a worst case of 100% GPS-reliant satellite, and no prior knowledge of previous satellite positions besides the one before it.

Location Tracking

- Devices like Apple Airtags don't need to be very accurate, so a 1 meter accuracy seems fit for these devices. If you are within this distance, you may be able to use other methods such as sound to locate your object.
- They don't need to be as frequent as with more mission-critical applications, but it is nice to have up-to-date information about your backpack if it is in the back of someone else's car, or stolen on the street, for example (rather dramatically).
  - Therefore, timing of every couple of seconds when something is moving fast, or adjusting to slower rates if it is not moving much

Stock Market

- Companies making trades as soon as the stock market changes is extremely important to "beat the market" and competitors.
- When multiple trades are near the same time, this allows the market to determine who bought and sold first, and adjust the rates accordingly such that every trade is counted to the ones before it with high accuracy.

- For trading, this is probably around milliseconds, but may be as low as microseconds due to extremely fast trading bots.
- Therefore, good atomic clocks should be used to synchronize the less precise clocks of the receivers

Outdoor Localization

- If a robot is traveling in an outdoor environment, it would like to know its location and velocity.
- The location accuracy needed would depend on the mission, or how close to objects the robot would like to get.
  - For example, if a robot has a "stay away from wall at least 10 cm" tolerance, than that would be the accuracy needed for GPS.
- If the robot has good dead reckoning with IMUs, or if it moves slowly, then it would require a smaller datarate. Therefore, the timing requirement could range anywhere between milliseconds to several seconds.

Self-care

Welcome to the start of the autumn quarter! While we hope you are all excited, it is also quite normal to feel nervous or anxious. It is always scary to start something new. In any case, it is important to acknowledge and welcome our feelings (the good *and* the bad) as well as be kind to ourselves.

What is one thing you have done over the past week that has helped you invite a sense of ease or even joy? Describe your experience in a few sentences. Alternatively, feel free to be creative in your form of expression, including but not limited to, a photo, video, or sketch.

Note: We do not want you to stress or overthink this exercise. Please engage in whatever way and to whatever extent you feel would best support you and your current needs. It can be as simple as a 5-minute walk, making a cup of tea, or taking a moment to admire the view from your window.

I went to the Arbor Live music jam night on Wednesday, where I played guitar with other people. I found some people to start a band with, and it cleared my head so I could get a lot of work done the rest of the week! We played "Dear Maria, Count Me In" by All Time Low and "Still into You" by Paramore.